

Lecture 301

Complex Number

Complex $c_1, c_2, x;$

$x = \text{cadd}(c_1, c_2);$

Example

$x = \text{cadd}(\text{cadd}(a, b), c);$

Operators

- The complete list of C++ operators that are overloaded is as follows

+ - * / % ^ &

! ~ ! = < > +=

-= *= /= %= ^= &= |=

<< >> >>= <<= == != <=

>= && || ++ -- --> *

-> [] () new new[] delete delete[]

Example

```
Return_type operator+(Argument_List)
{
```

// Body of function

```
}
```

Example

```
class Complex
```

```
{
```

```
private:
```

```
double real;
```

```
double imag;
```

// member function

};

Complex c1, c2;

c1 = c2;

Is equivalent to

c1.xreal = c2.xreal;

c1.imag = c2.imag;

Example

Complex Complex::operator+(Complex)

{

Complex temp;

temp.xreal = xreal + c.xreal;

temp.imag = imag + c.imag;

return temp;

}

complex number

 $z = x + d;$

complex number

Double Precision Number

اب اس کی definition ہے۔

Complex operator+(double d);

Friend Function

Date: 5/3

Friend Function in C++ is the creative function that breaks the data hiding in an object-oriented programming language.

Example

```
main()
```

```
{
```

```
Complex c1(1,2), c2(3,4), c3;
```

```
c3 = c1 + c2;
```

```
c1.display();
```

```
c2.display();
```

```
c3.display();
```

```
}
```

Complex operator + (Complex &c);

↓
C is a reference
to a complex
number

Example

Complex operator + (Complex &c1, Complex &c2)

```

}
Complex temp
temp.real = c1.getreal() + c2.getreal();
temp.imag = c1.getimag() + c2.getimag();
}

```

Example

```

class String

```

```

{

```

```

    private:

```

```

        char s[30];

```

```

    public:

```

```

        String ()

```

```

        {

```

```

            strcpy(s, "");

```

```

        }

```

// Declaration (prototype) of

overloaded sum operator String operator

```

+(String t);

```

```

}

```

پھر اس کو define میں دیا جائے۔

```

String String :: operator + (String t)

```

```

{

```

```
String temp;  
strcpy(temp, s);  
strcpy(temp, s);  
strcat(temp, s);  
return temp;  
}
```

Lecture 32

Example

```
void Date::operator++()  
{  
    if(day == daysOfMonth(day, month, year))  
    {  
        if(month < 12)  
        {  
            day = 1;  
            month++;  
        }  
        else  
        {  
            day = 1;  
            month = 1;  
        }  
    }  
}
```

```

    year++;
}
else
    day++;
}

```

Example

Data Data::operator + (int days)

```

{
    Data Temp;
    for (i=1; i < days; i++)
        ++Temp;
    return Temp;
}

```

days=5
↓
i < days

Bool → mean → true, false

Lecture 33

Example

```

Class String
{
    private:

```

```
char buf[100];
```

```
public:
```

```
String();
```

```
~String();
```

```
...
```

```
};
```

```
main()
```

```
{
```

```
String s1("This is a test");
```

```
String s2;
```

```
s2 = s1;
```

```
...
```

```
}
```

This pointer

```
buf;
```

```
this -> buf;
```

```
(*this).buf;
```

Self Assignment

```
main()
```

```
{
```

```
String s, *sPtr;
```

```
sPtr = &s;
```

Day: _____

```
s = *sptx;
```

```
}
```

Example

```
String String :: operator = (String  
                             &other)
```

```
{
```

```
if(this == &other)
```

```
return *this;
```

```
delete buf;
```

```
buf = new char[other.length + 1];
```

```
strcpy(buf, other.buf);
```

```
return *this;
```

```
}
```

Example

```
Date d1, d2;
```

```
d2 = d1++;
```

```
d2 = d1 + 1;
```

Conversion Function

```
Date()
```

```
{
```

```
//body of function
```

}

Lecture 34

Example

Fill the array

```
for (i = 0; i < arraySize; i++)
```

```
{
```

```
    cin >> c[i];
```

```
}
```

Print in Reverse Order

```
for (i = arraySize; i >= 0; i--)
```

```
{
```

```
    cout << c[i];
```

```
}
```

```
delete [] text;
```

Example

```
int *iptx;
```

```
iptx = new int[10];
```

```
delete iptx;
```

Overloading new
operator

```

void *operator new(size_t size)
{
    void *xtn = calloc(1, size);
    return xtn;
}

```

جب delete کرنے سے

```

void operator delete(void *ptx)
{
    free(ptx);
}

```

Example

```
int iArray[10];
```

```
int i;
```

```
for(i=0; i<100; i++)
```

```
{
```

```
    iArray[i] = i;
```

```
}
```

```
int iArray[10];
```

```
int i;
```

```
for(i=0; i<upperLimit; i++)
```

```
{
```

```
    iArray[i] = i;
```

}

Lecture 35

Stream

Stream is an ordered sequence of bytes

Stream Input/Output

Input stream object

cin

Output stream object

cout

Stream operators

Example

```
int i;
```

```
char c;
```

```
cin >> i;
```

```
cin >> c;
```

Every Stream has:

- A Source
- A destination

Day: _____

Date: 62

`cin.read(char* buffer, streamsize n)`

More than one

character is

read

Integer type to

express counts in

streams

Lecture 36

Number System

- Binary
- Octal
- Decimal
- Hexadecimal

Example

```
#include <iostream.h>
```

```
#include <iomanip.h>
```

```
main()
```

```
{
```

```
int i = 10
```

```
cout << oct << i << endl;
```

```
cout << hex << i << endl;
```

```
cout << dec << i << endl;
```

```
}
```

Output

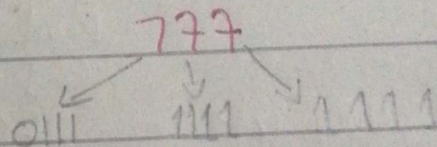
12

A

10

```
cout << setfill('*');
```

A character



Scientific Notation

1.2334e⁺⁰⁹

What we learnt so far...

- * Input Output Stream
- * And their Manipulations in C++.

Return type: Lecture 37
Reference to the output stream operator

ostream &operator << (ostream &output, vehicle A)
Reference to the output stream object of the class

Definition

ostream &operator << (ostream &oput, vehicle d)

{

output << d.seats;

output << d.tires;

return output;

}

Lecture 38

Use Define Manipulators

Example

```
int i = 10;  
cout << setw(7) << i << endl;
```

Definition

```
ostream & manipulatorName(ostream & os)
```

```
{  
    return os << userDefinedManipulator;  
}
```

User Defined Manipulators

A short Example

```
// Tab
```

```
ostream & tab(ostream & output)
```

```
{  
    return output << '\t';  
}
```

```
}
```

```
// bell
```

```
ostream & bell(ostream & output)
```

```
{  
    return output << '\a';  
}
```

```
}
```

Static

Are variables which exist for a certain amount of time which is longer than an

ordinary automatic variable.

Lecture 39

Pointers

A pointer is a special type of variable that contains a memory address.

- Void Pointer

- Pointer to an Integer

- Pointer to a Character

- Pointer to a Float

Dynamic Memory Allocation

```
int* P;
```

```
P = new int[10];
```

```
delete[] P;
```

```
m2 = m1;
```

```
int *m of m1
```

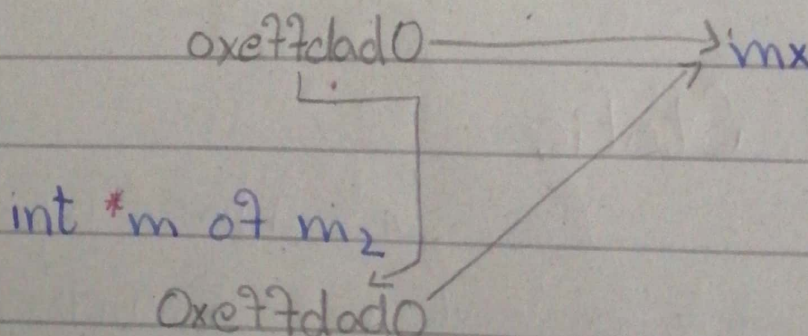
Pointer to the same
region in memory

0xeffcb0

mx

```
int *m of m2
```

0xeffd00



Rules

For dynamic memory allocation

- 1- Define copy constructor
- 2- Write assignment operator
- 3- Provide destructor

Lecture 40

Class

Class is a user defined data type.

Yes you can use user defined data type as a member of the class

Class can contain objects.

Example

```
class Person
```

```
{
```

```
private:
```

```
char *name;
```

```
char *address;
```

```
Data dateOfBirth;
```

```
// member object
```

```
public:
```

//public member function....

};